

AutoSkewBMT: Autonomously Synthesizing Optimized Integrity Authentication Mechanism for DNN Accelerators

Rakin Muhammad Shadab*, Sanjay Gandham†, Mingjie Lin‡

Department of ECE

University of Central Florida

Email: *ra397343@ucf.edu, †sanjay.gandham@ucf.edu, ‡Mingjie.Lin@ucf.edu

Abstract—As domain-specific accelerators for deep neural network (DNN) inference gain popularity due to their performance and flexibility advantages over general-purpose systems, the security of accelerator data in memory has emerged as a significant concern. However, the overhead associated with standard memory security measures, such as encryption and integrity authentication, presents a major challenge for accelerators, particularly given the high throughput demands of typical DNN applications [1].

In this work, we present AutoSkewBMT, a security framework that autonomously generates optimized integrity system configurations to enhance the Bonsai Merkle Tree (BMT)-based integrity authentication workflow for DNN accelerators. The framework leverages a novel and efficient design space generation algorithm to optimally skew the BMT for specific workloads. Configurations generated by AutoSkewBMT outperform recent state-of-the-art solutions by up to 32% on general DNN workloads.

Index Terms—DNN, accelerator, BMT, framework

I. INTRODUCTION

The age of Artificial Intelligence (AI) is upon us as the ever-growing, multi-faceted applications of machine learning and AI have brought about unforeseen levels of automation and advancements in different domains. In order to facilitate such massive growth, the hardware to accelerate machine learning and deep learning workflow has also gone through rapid development in the recent years. State-of-the-art, compute-focused Graphics Processing Units (GPUs) such as Nvidia's H100 graphics card can drive large-scale Deep Neural Network (DNN) models in servers and consumer-targeted spaces [2]. However, it might be impractical to use such massively sized GPU or GPU-like accelerators for all use-cases. For specialized applications that require domain-specific acceleration (including embedded and edge AI), it would be more imperative to use FPGA-driven DNN accelerators as they would typically perform better than GPUs in real-time inference, while also being a better fit for low-power, resource-constrained systems [3].

However, these custom DNN accelerators are susceptible to the same memory security exploits that typically affect the general-purpose computers [1], [4]. The common adversaries such as the confidentiality and integrity-focused threats can steal important information from the memory through a shared memory bus [5], [6]. The state-of-the-art techniques to mitigate these vulnerabilities such as AES encryption and BMT-based authentication (Fig. 1), provide strong security assurance but also incur high performance overhead [7]. Therefore, it is important to find the best balance between performance and security features, especially for accelerators as they need very high throughput for fast inference.

The contributions in this paper are mentioned below:

- This work proposes a new tool chain, AutoSkewBMT, for optimizing the integrity verification process in a secure DNN accelerator. Generally the integrity check is performed through an integrity tree such as BMT-based authentication [8], [9]. We strive to reduce the overhead of such authentication with

autonomous generation of the optimal configurations for such an integrity protection system for specific DNN workloads.

- As the accelerators operate with a fixed data access pattern, it is useful to skew the BMT to take advantage of the more-frequently accessed nodes. However, the skewing of BMT can be tricky especially for larger trees as the overhead might vary drastically for different skew patterns. AutoSkewBMT generates an optimally skewed BMT for an accelerator workload through the use of an efficient and novel design space generation & exploration algorithm based on Huffman encoding [10].
- AutoSkewBMT tool includes a counter access pattern generator that creates the metadata access patterns directly from the DNN model attributes and the accelerator parameters.

We limit ourselves to only BMT (excluding other integrity trees such as tree of counters (ToC) or parallel Merkle tree) for this work due to its extensive use in most of the contemporary literature [7], [11].

II. BACKGROUND & MOTIVATION

The memory-based vulnerabilities in a computing system can largely be divided into confidentiality and integrity-based attacks [11], [12]. The confidentiality-based threats attempt to directly access the user data over the memory bus [7]. The integrity-based threats, on the other hand, can be one of the three attacks – splicing, spoofing or memory replay – that overwrites an existing data block with another block of redundant data (Fig. 2) with spatial or temporal locality in the memory [6], [9]. The confidentiality of the memory contents can be protected by encrypting the entire memory with a state-of-the-art

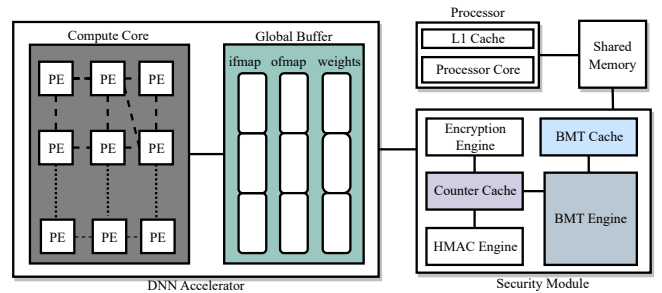


Fig. 1. A secure DNN accelerator with confidentiality and integrity protection.

encryption technique, such as the AES CTR-mode encryption [5], [8].

The AES CTR-mode encryption breaks down encryption metadata i.e., encryption counters into multiple smaller counters to generate a one-time pad (OTP) for the encryption of the data [11]. These counters or the encryption metadata are protected from the integrity threats by an integrity tree (Fig. 2). One of the most popular choice for the integrity tree is Bonsai Merkle Tree (BMT) as it has a low memory overhead and high flexibility of implementation [7], [9],

[13]. In such cases, the encrypted data can be integrity-protected through the use of a separate hash-based Message Authentication Code (HMAC) scheme [7], [11]. All of these security checks are performed within a trusted execution environment (TEE) outside the memory [9], [14].

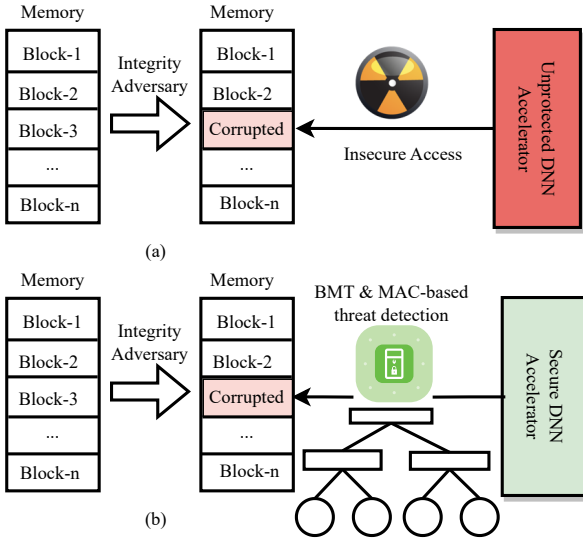


Fig. 2. (a) An unprotected DNN accelerator affected by integrity adversary, & (b) the same attack is detected by BMT & MAC in a secure accelerator.

TABLE I
COMPARISON BETWEEN RECENT SECURE DNN ACCELERATORS

	Confidentiality, Integrity	TEE	Tree/VN
Chiron [15]	✓	3rd-party	Tree
Occlumency [16]	✓	3rd-party	Tree
GuardNN [4]	✓	Custom	VN
TNPU [17]	✓	Custom	VN
Securator [1]	✓	Custom	VN

Since the DNN accelerators don't have enough on-chip memory to store the entire feature maps or weights, they must use the main memory (typically shared with the processor) for the data and instructions and as such, the same security threats also apply to them [1]. Therefore, they need similar TEE-based protection as the general-purpose systems, either from a third-party implementation [15], [16], [18] or a custom TEE [4], [17], [19]. However, the third-party TEE solutions (such as Intel SGX) are typically associated with high overhead and low flexibility/customization [1], [19]. As a result, more and more recent secure accelerators such as Securator [1], GuardNN [4] & TNPU [17] use their own, custom hardware-assisted TEEs. Since the accelerators are typically more bottlenecked by memory bandwidth than processors, these custom TEEs generally exclude the direct use of BMT or other integrity trees as they can incur high overhead for specific access patterns. Instead, they use alternate techniques such as Version Number (VN) to preserve integrity [1], [17]. But they either still need third-party integrity tree or additional TEE and software support from the processor to protect the VN (i.e., GuardNN & TNPU) and/or mandate close integration between the accelerator architecture and security module design, even embedding some security functionalities in the accelerator's

compute core (i.e., Securator). We strive to address this problem by allowing for a flexible, accelerator-independent security module (as seen in Fig. 1) and introducing a tool that can determine an optimized BMT subsystem configuration for a given accelerator to ensure high memory bandwidth.

In the threat model for this work, we assume the DNN accelerator, host processor & the security module to reside within a TEE. As such, they are considered secure. The memory is outside the TEE and therefore, is susceptible to the confidentiality & integrity attacks. We only consider the security of the accelerator data in this work and don't consider the use of non-volatile memory (NVM). Our target platform is FPGA due to its flexibility of re-configuration.

III. OVERHEAD IN BMT-BASED PROTECTION

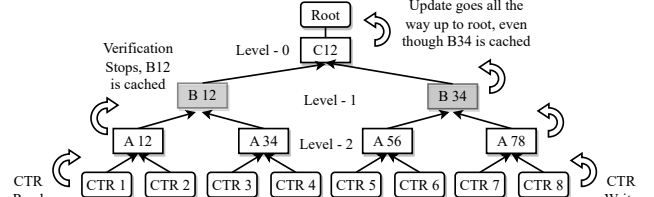


Fig. 3. BMT-based integrity protection.

The security module to prevent memory confidentiality and integrity-based attacks, as seen in Fig. 1, usually consists of AES encryption engine, a HMAC engine, a counter cache and a BMT engine + its associated BMT cache. Every memory access from the DNN accelerator must go through the security module for the security purposes. A BMT is constructed from the encryption counters where each of the counter values are hashed and then the results are truncated and concatenated to form a parent node. This process is repeated in the higher levels until the tree reaches only a single parent – root node. The number of children nodes for each parent node is determined by the so called 'arity' of the tree. For example, every parent node in a binary BMT (as shown in Fig. 3) has two children. The entire BMT except the root is stored in the memory (root is stored in a secure register).

Both the encryption and HMAC engines rely on the encryption metadata such as counters for the encryption and HMAC generation. A secure, on-chip counter cache is accessed to read/write a relevant counter value (Fig. 1). If the request to the counter cache is hit, then BMT authentication of the counter is skipped as the contents in the counter cache is considered secure. Otherwise, the counter is read from the memory, authenticated through BMT engine and then stored in the counter cache. In such cases, the counter request is forwarded to BMT engine that fetches the counter from the memory and verifies its integrity by re-generating its parent through hashing. If the parent node is not cached in the BMT cache and needs a memory access, then the parent needs to be verified as well. This triggers a chain verification process that only stops either at the root node or at a cached node (Fig. 3). We assume a dedicated BMT cache for each level of the BMT to enhance performance and reduce recursiveness in the BMT algorithms [7], [20]. Once the integrity is verified, the specific counter is then stored in the counter cache and can be used for future operations without re-verification unless it is evicted and read from the memory again. If the counter request is a write request, then the counter is incremented (and authenticated through BMT first, if missed in the cache) and its corresponding BMT chain also needs to be updated. This update can be done immediately/eagerly or in a relaxed manner [7], [9], but we will only consider the eager update

process in this work due to its widespread usage. The eager-update technique always updates the entire chain regardless of the caching status of the BMT nodes (Fig. 3). The BMT authentication and update process, while thwarting the previously mentioned integrity attacks, also incurs significant overhead.

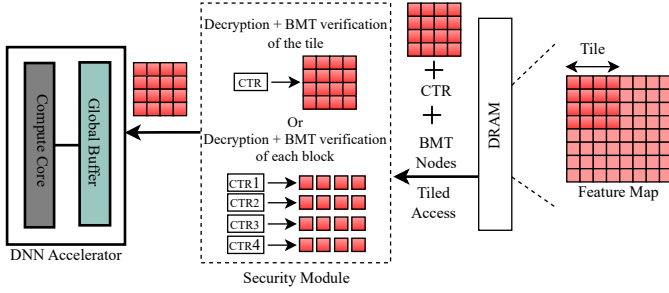


Fig. 4. Tile-level vs block-level integrity authentication.

DNN accelerators use tiling technique to access feature maps and weights of the DNN model from the memory. While utilizing tiled accesses, the accelerator only fetches a section of the feature map in its global buffer, known as a tile. This is done due to the local buffer size constraints on the accelerator. An integrity-protected accelerator (such as an accelerator with BMT protection) with tiled accesses can use encryption counters to protect the entire tile (tile-level integrity protection) or they can be used to encrypt and protect individual data blocks (block-level integrity protection). The tile-level integrity protection can lower the integrity overhead since only one counter protects the entire tile. In such cases, the counter is only updated once all the data blocks within the tile are written or updated. During tiled data read, only one counter needs to be verified by the BMT since each counter would protect an entire tile (Fig. 4). However, this type of protection is not crash-consistent and cannot provide security guarantee for every block in the tile during a system crash/power failure if the system memory is Non-Volatile Memory (NVM). On the other hand, the block-level integrity protection is the same type of protection used in general computing systems. While this technique can cause additional overhead due to the update/authentication of individual blocks, it is flexible and can be used with any type of memory.

Since the DNN accelerators operate on a fixed and predictable data access pattern, the BMT-authentication and update process can benefit from making the verification and update path of some of the most commonly accessed counters shorter compared to other, less frequently-accessed counters. This is known as the skewing of the BMT (Fig. 5). Let's assume that each of the counters except *CTR4* and *CTR8* have an access frequency of 2 (i.e., they are accessed twice) and *CTR4* and *CTR8* are more frequently accessed with an access frequency of 8. If the accesses are write accesses, then the balanced BMT (Fig. 3) will need a hash operation at each level, totalling 4 hashes (including root) for a write to a counter. Therefore, the total number of hash operations for the balanced BMT = $6*4*2 + 2*4*8 = 112$. On the other hand, the same scenario for the skewed BMT in Fig. 5 requires $(4*5*2 + 2*4*2 + 2*2*8) = 88$ hash operations.

Even though FAST [21] already used skewed BMT for processors, it didn't use a combined counter access pattern generator in the tool chain and instead collected the counter access traces for each application by separately simulating them. It also didn't use a BMT cache and relied on the use of direct BMT node access from the memory. Moreover, the memory access patterns and the granularity of

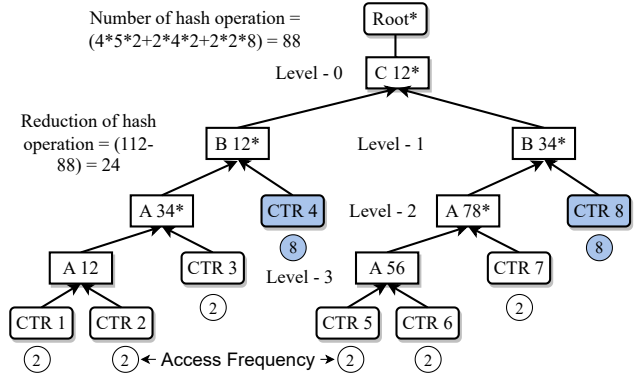


Fig. 5. An example of a skewed BMT with the same counters as in Fig. 3. Here, *CTR4* and *CTR8* are the most frequently accessed and placed closer to the root. The * sign indicates a change in the hash value compared to the balanced BMT shown in Fig. 3.

integrity authentication in DNN accelerators depend not only on the workload but also on the accelerator design features such as tiling (as shown in Fig. 4). FAST algorithm doesn't account for these scenarios as it is only targeted for general-purpose applications.

In the case of a DNN accelerator, the parameters are fixed at the system initialization, the memory accesses are also fixed for a given DNN model. Therefore, the skewed BMT configuration will also be constant for that specific accelerator workload. As such, we propose a security framework, AutoSkewBMT, that determines a skewed BMT configuration for a set of accelerator parameters and a fixed DNN model. The counter access pattern is also generated automatically from the DNN model attributes and accelerator parameters, eliminating the need to determine it separately by running inference on an accelerator. If DNN model/workload changes, the skewed BMT will be re-skewed through the tool chain for the new workload instead of dynamic, run-time reformation.

IV. AUTOSKEWBMT: METHODOLOGY

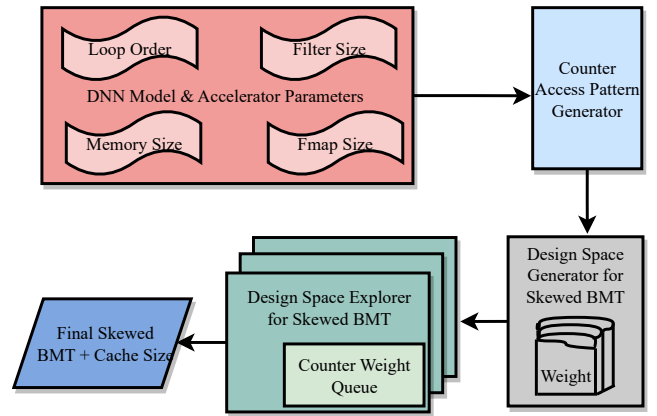


Fig. 6. AutoSkewBMT framework.

The AutoSkewBMT framework consists of three components - counter access pattern generator, design space generator and design space explorer (Fig. 6). The DNN model parameters such as the input and output feature map size, filter size, memory size and accelerator parameters such as global buffer size and loop order are provided as the input to the framework. These attributes are then used to generate the data access pattern for the accelerator data and then consequently, the resultant counter access pattern to encrypt/decrypt the data block.

The counter accesses are then fed into a design space generator that generates different weights for every counter, based on the frequency of read and write accesses. These weights are then passed over to the design space explorer to determine an optimized skewed BMT structure by placing the most frequently accessed counters closest to the root node. It also fixes an optimal BMT cache size at each tree level of the skewed BMT based on the number of nodes at each level. The output of our tool is a skewed BMT configuration optimized for a fixed accelerator design and a specific DNN workload.

Algorithm 1: Counter Access Pattern Generation

Input: Memory Size S , input size I_S , output size O_S , filter size F_S , loop order R , global buffer size B_S

Output: List of Counter Accesses, L_C

Parameters: Feature map dimension: x, y, z , tile dimension: t_x, t_y, t_z , filter co-efficient: n , initial memory address $base_{addr}$, current data address $addr$, initial counter address $base_{ctr}$, counter address ctr_{addr} , list index i

```

while  $Access_{req} < S$  do
  Calculate  $t_x, t_y, t_z$ ;
  Calculate  $addr =$ 
     $base_{addr} + x + (y * t_x) + (z * t_x * t_y) + (n * t_x * t_y * t_z)$ ;
  Update  $ctr_{addr} = base_{ctr} + (addr/2^{12})$ ;
  Append  $L_C[i] = ctr_{addr}$ ;
  Increment  $i$ ;
  Increment  $Access_{req}$ ;
  Update  $x, y, z, n, t_x, t_y, t_z$ ;
end

```

Algorithm 2: Design Space Generation & Exploration

Input: List of Counter Accesses with the number of read and write request, L_C , size of the protected memory S , base address of the counter range, $base_{ctr}$

Output: Shape of the optimized skewed tree and the size of the BMT cache

Parameters: Number of children nodes for each parent or the -arity of the BMT, K , an empty weight queue, Q , a list for the weights of every access, W , list index, j , number of read accesses, ctr_{rd} , number of write accesses, ctr_{wr} , skewing index, p , skewing threshold, T , number of BMT levels, M , tile update/verification weight, $T_{u/a}$

```

for  $L_C[i]$  in  $L_C$  do
   $j = ctr_{addr} - base_{ctr}$ ;
   $W[j] = W[j] + (ctr_{rd} + (M+1) * ctr_{wr}) * T_{u/a}$ ;
  Push  $W[j]$  into  $Q$ ;

```

end

while Q is not empty & $p \leq T$ **do**

```

  Pop  $K$  entries from  $Q$ ;
  Update the new parent of the popped entries to be  $N$ ;
  Push  $N$  with the popped entries;
  increment  $p$ ;

```

end

Append the remaining nodes;

Fix cache size at each BMT level = total cache size * % of total nodes at the level;

The counter access pattern generator calculates the current data address, $addr$ from the accelerator parameters (Alg. 1). The accelerator typically accesses a feature map in a group of tiles. As the dimensions of the feature maps might vary for different layers of a DNN model, the tile dimensions would also change. As such, these dimensions will be different in accordance with the current layer of the DNN and the model itself. Once $addr$ is generated, the counter address, ctr_{addr} is calculated from the $addr$. The base address of the counter, ctr_{base} typically starts after the data and the HMAC address range in the memory. The counter address, ctr_{addr} is later appended into a list L_C that also includes the information on the request type (read/write). This list is forwarded to the design explorer tool at the end of the access pattern generation (Fig. 6).

The design space generator tool calculates a weight for each counter request based on the information provided by the access pattern generator. These weights are based on the worst-case scenario for a read or write operation. In the worst-case, a counter read request will result in a BMT verification chain that goes all the way up to the root since none of the nodes will be present in the BMT cache. However, once the nodes are read, the verification of all the nodes in the chain can be done concurrently using parallel hash engines, as seen in [14] & [20]. On the other hand, a counter write will always go through the entire update chain (Fig. 3) as we consider an eagerly updated tree. We assume a BMT with 5 levels ($M = 5$) for the context of this work, and as such, the counter write operation will require $M + 1 = 6$ consecutive, non-overlapped hash operations to update a node at each level, including the sixth level (root). However, the algorithm is valid for a BMT with any number of tree levels as M can be any valid integer number. In other words, it is scalable with the size of the tree.

We assign a weight of 1 to each read and a weight of $M + 1$ (6 in this case) to each write to a counter (Alg. 2). The total weight is then calculated by summing these two weights and multiplied with a weight of tile update/authentication granularity ($T_{u/a}$). $T_{u/a}$ is calculated by the ratio of total number of verification/updates required in a tile to total number of uncached counter accesses in a tile. For block-level integrity protection, this weight is simply going to be 1. The counters with the highest total weight will then be put closest to the root node in the final skewed BMT. These weights are used by the design space explorer tool to generate the skewed BMT. The weights are stored in a priority queue (called weight queue) and their parents are updated accordingly to skew the verification/update chain (Alg. 2). Based on a skewing threshold, the number of counter nodes being pushed up is decided and all the remaining counters are placed at the bottom of the tree while the rest of the nodes are appended. The size of the cache at each level of the BMT is then fixed, based on the total percentage of nodes at that level. The algorithm for design space generation & exploration (Alg. 2) is inspired by popular Huffman encoding [10], similar to FAST [21].

The AutoSkewBMT toolchain relies on memory access pattern and parameters like feature map size and the number of accesses in a tile. But it doesn't have any dependency on the actual type of computation in the each layer of the DNN model (i.e., convolution or matrix-matrix multiplication) or the computation techniques in the accelerator. Therefore, it is highly flexible and can work with any type of DNN models and accelerators if the aforementioned parameters are known.

Our proposed work can be used for both tile-level and block-level integrity granularity. However, for the scope and evaluation of this work, we only use block-level granularity.

V. CONSISTENCY OF SECURITY GUARANTEE WITH AUTO-SKEWBMT

The skewed BMT generated by AutoSkewBMT framework follows the same authentication and update algorithm as a regular BMT. The only difference is that the values of the parents for the nodes that are moved to higher levels will be different than the original BMT as the accompanying nodes under the same parent will be different from the original orientation (as seen in Fig. 5). However, this change in the hash values within an authentication/update chain doesn't violate the security assurance as the root will also be updated accordingly and the entire authentication/update chain will maintain the consistency with the new values. In other words, the security guarantee in the skewed BMT will remain the same as the regular BMT.

VI. EVALUATION

We compare three different accelerator systems – a system with no security protection (baseline), one with both encryption-based confidentiality protection and a balanced BMT-based integrity protection and finally the system with encryption and skewed BMT (configured by AutoSkewBMT) – against each other. We also include two other state-of-the-art solutions – GuardNN [4] & TNPU [17] – in the mix for the performance analysis. We consider the use of a 5-level balanced BMT that is skewed for different access patterns by the AutoSkewBMT tool. Both the counter cache and the BMT caches are direct-mapped caches. We utilize a state-of-the-art BMT engine architecture (similar to HERMES [14]). The configuration of the security module is elaborated on Tab. II.

TABLE II
SECURITY MODULE CONFIGURATION

Encryption, MAC, Counter cache & BMT engine details	
Protected memory size	128MB
Encryption	AES-128
MAC	HMAC (SHA-256 Hash)
Data granularity	64B
HMAC block size	64B
BMT parameters	5-level, 8-ary
Counter cache size	32KB
BMT cache size	40.3KB (32KB+4KB+4KB+128B+128B)

The security blocks are synthesized for Xilinx U200 FPGA platform with Vivado tool (Tab. III). The BMT engine designed for the integrity authentication with the resultant skewed BMT configuration from AutoSkewBMT incurs only a 3% overhead for the Look-Up Table (LUT) usage and a negligible overhead of flip-flop usage over the BMT engine for the regular, balanced BMT. Our energy analysis shows less than 2% on-chip power increase for the system configured by the AutoSkewBMT over the balanced BMT. We also synthesized the security modules for ASIC using Synopsys DC and found similar 3% overhead for the skewed BMT engine.

We compare the performance of all three of our systems with TNPU [17] & GuardNN [4]. The optimized and shortened BMT authentication chain for most frequently accessed counters allows the AutoSkewBMT system to exhibit up to 7% better performance compared to TNPU and 8% better performance over the balanced BMT in widely used DNN workloads (Fig. 7). TNPU depends on

TABLE III
HARDWARE SYNTHESIS RESULTS

	LUT	Flip-flop	BRAM
Encryption Engine	36997(3.1%)	13038(0.6%)	0(0.0%)
HMAC Engine	6205(0.5%)	6133(0.3%)	0(0.0%)
Counter Cache	3391(0.3%)	4019(0.2%)	8.5(0.0%)
BMT Engine	137650 (11.7%)	158300 (6.7%)	14.5 (0.7%)
Skewed BMT Engine	172540 (14.7%)	164700 (6.9%)	14.5 (0.7%)
BMT Cache (Total)	48912 (4.1%)	40447 (1.71%)	128 (5.9%)

a third-party integrity tree (such as Intel SGX) support for the protection of its integrity metadata [17] and exhibits some overhead because of it. Our system also outperforms GuardNN by up to 32% margin as GuardNN suffers from additional overhead caused by a software-based scheduler and processor TEE for its integrity mechanism [4].

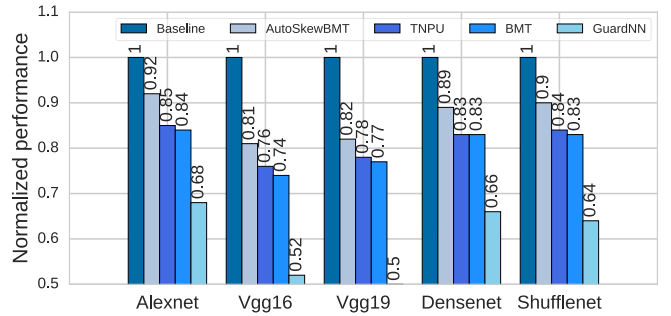


Fig. 7. Normalized performance comparison.

The distribution in performance is supported by Tab. IV. As the re-organization of the counters within the skewed BMT is heavily influenced by the number of writes (as tree updates due to counter write is the critical bottleneck, as discussed in the previous section), the resultant lower number of hash operations would directly coincide with the performance boost. As a consequence, we see the best performance improvement for AutoSkewBMT in Alexnet (Fig. 7) as AutoSkewBMT optimization results in 23% reduction of the total number of hash operations for that workload (Tab. IV).

TABLE IV
NUMBER OF HASH OPERATIONS FOR COUNTER WRITE

	Alexnet	Vgg16	Vgg19	Densenet	Shufflenet
BMT	65352	4005648	4503744	140928	207366
Skewed BMT	50376	3456734	4018253	121762	193397
Reduction	23%	13.7%	10.8%	13.6%	6.7%

Even though the total number of writes to different counters is only about 20% of the total counter operations for all workloads (Tab. V), the writes should have a high impact on the system performance even for balanced BMT as every data write i.e., counter write must initiate an update to the tree in an eager-update system (even for a write hit in the counter cache, as every write must be persisted to the tree). The update of the tree due to a counter write can take at least six

times higher latency than a verification of the said counter during a read (as mentioned before). It is because the hashing of all the nodes can be performed in parallel during the read operation after the nodes are read, while the hashing during the update operation for an eagerly-updated tree needs to be performed in sequence and cannot be overlapped [7], [20]. This difference can be even larger for BMT cache hits since the verification always stops at a cached node, while the update is always propagated to the root for eager update (as seen in Fig. 3). Since both the Vgg models use a significantly higher number of counters and read/write operations compared to other models, all the secure systems exhibit higher overheads for these models compared to other workloads. On the other hand, Densenet and Shufflenet have a higher overhead for all the systems compared to the Alexnet even though they have relatively lower number of total counter operations (Tab. V). However, both of them also have considerably higher number of counter write operations compared to Alexnet (Tab. V), which explains the additional overhead.

TABLE V
COUNTER ACCESS STATISTICS

	No. of Counters	Counter Read	Counter Write
Alexnet	700	45052	10892
Vgg16	6291	2153945	667608
Vgg19	7573	2867225	750624
Densenet	237	3612	23488
Shufflenet	320	3686	34561

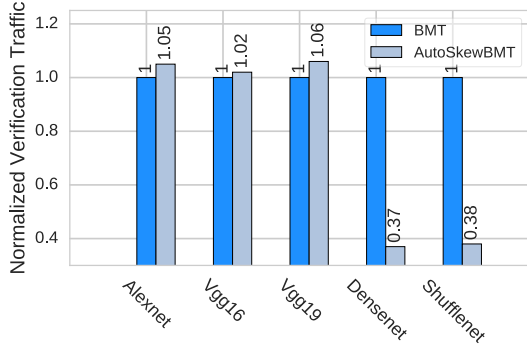


Fig. 8. Normalized worst-case BMT verification.

We again initiate the workloads, but this time without the BMT caches to determine the total number of worst-case BMT node verification operations due to counter read requests. In the absence of BMT cache, every single BMT node in a verification chain needs to be read from the memory and verified individually. This process will always reach the root in this case. As AutoSkewBMT puts significantly more weights on the counters with higher number of writes compared to the number of reads (Alg. 2), some counters that are more frequently accessed for writes would get prioritized for skewing over the counters with a high number of reads but no writes as these counters with read-heavy accesses might be pushed down to lower levels of the tree. This would nominally increase the number of node verification from some workloads (Fig. 8) as the counters at the bottom-most level would now have to traverse one extra tree level (as seen with CTR1, CTR2, CTR5 and CTR6 in Fig. 5). However, both Densenet and Shufflenet generate read accesses to only a few

out of all the counters. This scenario allows these counters to have a large weight as they also have high number of writes and as such, they are moved up the tree level during the skewed BMT generation. As a result, we see a drastic reduction of the percentage of total node verification for Densenet and Shufflenet in this simulated worst-case scenario (Fig. 8). The latency breakdown in Fig. 9 shows the largest BMT latency reduction for Alexnet (as it reduces the hashing by 23% (Tab. IV)), but all the benchmarks benefit from AutoSkewBMT. The latency breakdown in Fig. 9 shows the largest BMT latency reduction for Alexnet (as it reduces the hashing by 23% (Tab. IV)) and results in best performance gain in Fig. 7), but all the benchmarks benefit from AutoSkewBMT. Even though both Densenet and Shufflenet have a lot of write operations (Tab. V), they see less latency reduction as the percentage of reduced hash operations for them is relatively lower (Tab. IV).

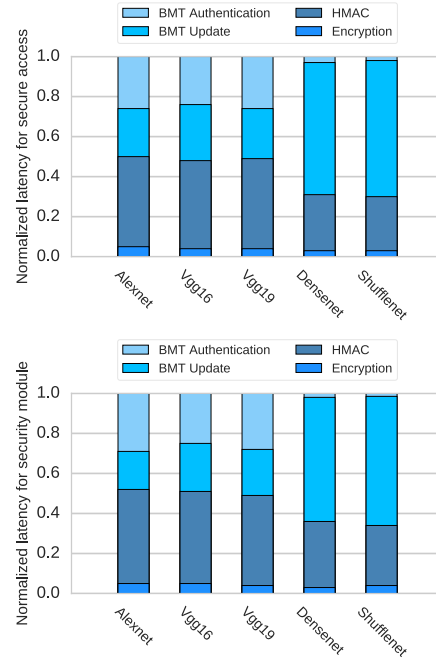


Fig. 9. Normalized latency for different security during memory access, for a secure-memory system with regular BMT (top) and the system with AutoSkewBMT-generated BMT configurations (bottom).

VII. DISCUSSION

AutoSkewBMT can generate an optimally skewed BMT configuration for a particular DNN workload and accelerator before the system is initialized. It is flexible, scalable and can be used with any DNN workload. However, currently there is no run-time tree regeneration support. Every time a new model is used, the toolchain needs to be compiled again with the new parameters to create a new version of the tree. Future research might look into making the run-time tree generation process feasible with low overhead.

VIII. CONCLUSION

We propose AutoSkewBMT, a tool chain to autonomously generate the configurations for an optimized metadata integrity authentication system for FPGA-based DNN accelerators. Our tool generates an optimal integrity tree from the design parameters for an accelerator and the feature map details of a given DNN model. We observe up to 7% and 32% respective performance improvement for our work over two state-of-the-art secure accelerator setups.

REFERENCES

- [1] N. Shrivastava and S. R. Sarangi, "Securator: A fast and secure neural processing unit," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 1127–1139.
- [2] A. C. Elster and T. A. Haugdahl, "Nvidia hopper gpu and grace cpu highlights," *Computing in Science & Engineering*, vol. 24, no. 2, pp. 95–100, 2022.
- [3] P. Dhilleswararao, S. Boppu, M. S. Manikandan, and L. R. Cenkeramaddi, "Efficient hardware architectures for accelerating deep neural networks: Survey," *IEEE Access*, 2022.
- [4] W. Hua, M. Umar, Z. Zhang, and G. E. Suh, "Guardnn: secure accelerator architecture for privacy-preserving deep learning," in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, 2022, pp. 349–354.
- [5] A. Awad, M. Ye, Y. Solihin, L. Njilla, and K. A. Zubair, "Triad-NVM: Persistency for integrity-protected and encrypted non-volatile memories," in *Proceedings of the 46th International Symposium on Computer Architecture*, 2019, pp. 104–115.
- [6] R. Elbaz, D. Champagne, C. Gebotys, R. B. Lee, N. Potlapally, and L. Torres, "Hardware mechanisms for memory authentication: A survey of existing techniques and engines," *Transactions on Computational Science IV*, pp. 1–22, 2009.
- [7] Y. Zou, K. A. Zubair, M. Alwadi, R. M. Shadab, S. Gandham, A. Awad, and M. Lin, "ARES: Persistently secure non-volatile memory with processor-transparent and hardware-friendly integrity verification and metadata recovery," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 21, no. 1, pp. 1–32, 2022.
- [8] A. Freij, H. Zhou, and Y. Solihin, "Bonsai Merkle forests: Efficiently achieving crash consistency in secure persistent memory," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 1227–1240.
- [9] R. M. Shadab, Y. Zou, S. Gandham, A. Awad, and M. Lin, "HMT: A hardware-centric hybrid bonsai merkle tree algorithm for high-performance authentication," *ACM Transactions on Embedded Computing Systems*, vol. 22, no. 4, pp. 1–28, 2023.
- [10] D. A. Huffman, "A method for the construction of minimum-redundancy codes," *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, 1952.
- [11] A. Awad, S. Suboh, M. Ye, K. A. Zubair, and M. Al-Wadi, "Persistently-secure processors: Challenges and opportunities for securing non-volatile memories," in *2019 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*. IEEE, 2019, pp. 610–614.
- [12] M. Henson and S. Taylor, "Memory encryption: A survey of existing techniques," *ACM Computing Surveys (CSUR)*, vol. 46, no. 4, pp. 1–26, 2014.
- [13] R. M. Shadab, Y. Zou, S. Gandham, and M. Lin, "OMT: A run-time adaptive architectural framework for bonsai merkle tree-based secure authentication with embedded heterogeneous memory," in *2023 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. IEEE, 2023, pp. 191–202.
- [14] Y. Zou, A. Awad, and M. Lin, "Hermes: Hardware-efficient speculative dataflow architecture for bonsai Merkle tree-based memory authentication," in *2021 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. IEEE, 2021, pp. 203–213.
- [15] T. Hunt, C. Song, R. Shokri, V. Shmatikov, and E. Witchel, "Chiron: Privacy-preserving machine learning as a service," *arXiv preprint arXiv:1803.05961*, 2018.
- [16] T. Lee, Z. Lin, S. Pushp, C. Li, Y. Liu, Y. Lee, F. Xu, C. Xu, L. Zhang, and J. Song, "Occlumency: Privacy-preserving remote deep-learning inference using sgx," in *The 25th Annual International Conference on Mobile Computing and Networking*, 2019, pp. 1–17.
- [17] S. Lee, J. Kim, S. Na, J. Park, and J. Huh, "Tnpu: Supporting trusted execution with tree-less integrity protection for neural processing unit," in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, pp. 229–243.
- [18] H. C. Tanuwidjaja, R. Choi, S. Baek, and K. Kim, "Privacy-preserving deep learning on machine learning as a service—a comprehensive survey," *IEEE Access*, vol. 8, pp. 167 425–167 447, 2020.
- [19] W. Hua, M. Umar, Z. Zhang, and G. E. Suh, "Mgx: Near-zero overhead memory protection for data-intensive accelerators," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 726–741.
- [20] R. M. Shadab, Y. Zou, S. Gandham, A. Awad, and M. Lin, "A secure computing system with hardware-efficient lazy bonsai merkle tree for fpga-attached embedded memory," *IEEE Transactions on Dependable and Secure Computing*, 2023.
- [21] Y. Zou and M. Lin, "Fast: A frequency-aware skewed Merkle tree for FPGA-secured embedded systems," in *2019 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*. IEEE, 2019, pp. 326–331.